

From Console Quiz to Adaptive AI Interview Panel

A complete engineering plan for rebuilding the AI-Interview project on Agora Conversational AI — gap analysis, target architecture, a zero-budget stack, code-reuse disposition, and a day-by-day build schedule through the Grand Finale.

REPOSITORY	AI-Interview — branch <code>main</code> @ <code>f03c143</code>
PROBLEM STATEMENT	PS11 — Adaptive voice interview platform
PLATFORM	Agora Conversational AI Engine (mandatory)
BUDGET	Zero — free tiers only
DOCUMENT DATE	19 August 2026
SUBMISSION DEADLINE	6 September 2026, 11:00 PM IST
GRAND FINALE	12 September 2026, Delhi (offline)

§ Contents

1. Executive Summary

The verdict, the three decisions that define the project, and the effort estimate.

2. What the Hackathon Actually Requires

Two independent bars, the PS11 capability list, evaluation criteria, and the timeline.

3. Current State Audit

Module inventory, line counts, capability gap matrix, and defects found in the existing code.

4. Target Architecture

Why the voice stack is retired, the OpenAI-compatible bridge, floor control, tool calling, and the latency budget.

5. Zero-Budget Stack & Quota Discipline

What each component costs, the agent-minute budget that governs the project, provider fallback, and the trade-offs accepted.

6. Code Reuse Disposition

File-by-file verdict, target repository structure, and what changes in each ported module.

7. The Roadmap

Team lanes, the integration contract, and a day-by-day sprint plan with acceptance conditions.

8. Demo Script & Submission Checklist

The four-minute demo narrative and everything Round 4 requires.

9. Risk Register

What can go wrong, early warning signs, and mitigations.

10. Appendices

Agora API reference, implementation recipes, environment variables, persona definitions, sources.

1 Executive Summary

The existing project is a competent single-interviewer console quiz. PS11 asks for a real-time, interruptible, multi-persona voice panel built on a platform the project has never touched. This is a re-architecture with substantial code reuse — not an extension, and not a rewrite from zero.

1.1 The verdict

Measured against the two bars the hackathon sets, the project currently clears roughly **1.5 of 11 PS11 capabilities** and **0 of 1 mandatory platform requirements**. The mandatory requirement is the binding one: the guidelines state that projects *"must use Agora Conversational AI as a core component of the real-time voice experience,"* and "Use of Agora Conversational AI" is scored as its own criterion. A submission without it is not a weak submission — it is an ineligible one.

The good news is that the codebase is small (605 lines of active code) and the part that is genuinely hard to build — rubric-driven answer evaluation — is the part that survives. What has to go is the input/output layer, which is exactly the layer Agora replaces.

1.2 The three decisions that define this project

DECISION 1 — RETIRE THE LOCAL VOICE STACK

`audio_recorder.py` and `whisper_asr.py` (126 active lines) are deleted, not adapted. Agora Conversational AI is the ASR, VAD, barge-in, turn-detection and TTS pipeline. Running Whisper alongside it is duplication that judges will read as a bolt-on, and the record-until-silence model is the specific thing that makes interruption impossible today.

DECISION 2 — THE INTERVIEW BRAIN BECOMES AN OPENAI-COMPATIBLE HTTP SERVICE

Agora's `llm.url` accepts any endpoint implementing OpenAI's `/chat/completions` contract (`"style": "openai"`). That is the bridge. Your conductor, rubrics, and scoring move behind that interface and Agora drives them. Because the free LLM providers in §5 are themselves OpenAI-compatible, the gateway is a near pass-through with your logic injected — no message-format translation layer is required.

DECISION 4 — ENGINEER AROUND AGENT-MINUTES, NOT DOLLARS

The project runs entirely on free tiers. Every component has one except Agora, which is mandated by the rules and meters **300 free agent-minutes per month** — ten three-persona interviews for the entire month, covering development, testing, rehearsal, the video, and the live demo. A mock replay harness keeps roughly 80% of the build off the metered path, and `PANEL_SIZE` is a runtime flag so the panel can drop from three personas to two without a code change. Full treatment in §5.

DECISION 3 — ONE AGORA AGENT PER PERSONA, GATED BY A CONDUCTOR

Three agents join one RTC channel with three distinct TTS voices. All are subscribed to the candidate, so all call your gateway when the candidate stops speaking. The Conductor decides who holds the floor: the floor-holder streams a real reply, the others return an empty completion and stay silent. Turn-taking policy lives entirely in your code, uses no undocumented behaviour, and is demonstrable on stage.

1.3 Effort estimate

WORKSTREAM	NEW LINES (EST.)	PORTED (EST.)	DIFFICULTY	OWNER
Agora session & agent lifecycle	250	0	Medium — new API surface	Dev A
Gateway (SSE) + provider fallback	300	0	Medium — streaming correctness	Dev A
Mock replay harness	120	0	Low — but build it first	Dev A
Conductor & floor control	350	25	High — the core IP	Dev B
Analysis workers & ledgers	400	95	Medium — mostly ported	Dev B
Final assessment & report	200	30	Low	Dev B
Web client & UI	450	0	Medium — Agora Web SDK	Dev C
Total	2,070	165		

Roughly **165 of 605 existing lines carry forward (27%)**, almost all of it inside `src/back_end/evaluation/`. The development sprint runs 29 August – 3 September, which is six working days; the estimate above is achievable for a team of three, and tight for a team of two.

WHY THE REUSE RATIO IS THE RIGHT ONE

The guidelines permit prior work as a foundation but require that "*significant development and innovation must take place during the hackathon.*" Carrying forward the evaluation engine is a legitimate head start on the part of PS11 most teams will handwave, while the panel, conductor, and Agora integration are all built during the sprint. That is a clean story for the mandatory disclosure.

2 What the Hackathon Actually Requires

There are two independent bars, and the project is in very different shape against each. Separating them prevents the common failure of polishing PS11 features while remaining ineligible on the platform requirement.

2.1 Bar A — Platform requirements (apply to every team)

REQUIREMENT	WHAT IT MEANS IN PRACTICE	STATUS
Agora Conversational AI as a core component	Not a peripheral integration. The real-time voice experience must run through it.	Absent
Working prototype / functional demo	Judges must be able to open a link. A local CLI requiring Ollama and a downloaded Whisper model does not qualify.	Absent
Meaningful conversational AI, not a text chatbot	Explicitly called out twice in the guidelines.	Partial
GitHub repository	Public, documented, reproducible.	Have
Demo video	Required at Round 4.	Absent
Architecture / tech-stack document	Required at Round 4.	Absent
Written explanation of Agora integration	Called out as its own submission line item. Make it a top-level README heading, not a buried paragraph.	Absent
Disclosure of pre-existing work	Required where an existing project is used as a foundation.	Absent
Team of 2–4 members	Minimum two. Solo entries are not eligible.	Verify

2.2 Bar B — PS11 capability checklist

The problem statement lists eleven capabilities the solution "should demonstrate." Treat this as a scoring rubric — build one demonstrable moment per line.

#	CAPABILITY	WHAT A JUDGE NEEDS TO SEE
1	Real-time and interruptible voice interviews	Candidate talks over the interviewer; the interviewer stops mid-word and yields.
2	Multiple interviewer roles or personalities	Audibly distinct voices with distinct concerns, not one voice announcing role names.
3	Shared candidate context between roles	Persona B references something the candidate told persona A, unprompted.
4	Dynamic follow-up questions	The follow-up is visibly derived from the answer, not a scripted next item.
5	Controlled interviewer turn-taking	Exactly one interviewer speaks at a time; handoffs are deliberate and explainable.
6	Role-play or scenario-based questions	An interviewer stays in character inside a scenario across several turns.
7	Difficulty adjustment based on performance	A visible difficulty level that moves in response to answer quality.
8	Identification of vague or contradictory answers	The system catches a self-contradiction across turns and challenges it.
9	Evidence-based feedback linked to the transcript	Every claim in the report cites a turn ID, timestamp, and verbatim quote.
10	A structured final assessment	Per-competency scores, strengths, gaps, and a recommendation.
11	Clear disclosure that the candidate is interacting with AI	Spoken at session start and persistently visible in the UI.

THE EXAMPLE SCENARIO IS A GIFT — BUILD THE DEMO AROUND IT

PS11 supplies its own demo script: "A candidate provides a technically correct solution but does not explain its impact on customers. The technical interviewer may accept the implementation, while the product interviewer should challenge the candidate to explain the business implications."

Reproducing this exact moment on stage demonstrates capabilities 2, 3, 5 and 6 simultaneously, and signals to judges that you read the statement carefully. Make it the centrepiece of the demo video.

2.3 Evaluation criteria — and how they shift between rounds

Round 4 and Round 5 are judged on different axes. Budget your preparation accordingly: the online evaluation rewards engineering, the finale rewards commercial framing.

ROUND 4 — ONLINE EVALUATION (4–6 SEP)	ROUND 5 — GRAND FINALE (12 SEP)
Innovation & creativity	Scalability
Use of Agora Conversational AI	Business potential
Technical implementation	Real-world impact
Quality of voice / conversational experience	Quality of live demonstration
Real-world impact & usefulness	<i>Shark Tank</i> –style pitch before an expert jury. At least one team member must present in person.
Product readiness & scalability	

2.4 Timeline and hard deadlines

ROUND	DATES	STAGE AND WHAT IT DEMANDS OF YOU
1	21 Jul – 25 Aug	Registration & team formation via Commudle. Submit problem statement, tech stack, track. Six days from today.
2	26 – 28 Aug	Idea submission & screening. Deck covering problem/solution, technical architecture, expected impact, and Agora integration plan. Top 250–500 advance.
3	29 Aug – 3 Sep	Development sprint. One dedicated 1:1 mentorship session; Agora support on Discord. Six working days — this is the entire build window.
4	4 – 6 Sep	Project submission & online evaluation. Deadline 6 Sep, 11:00 PM IST. Repo, prototype, demo video, pitch deck, live online presentation.
5	8 – 12 Sep	Finale prep and Grand Finale, Delhi. No major features may be added after Round 4 — UI/UX and presentation polish only.

READ THE FEATURE FREEZE CAREFULLY

Everything that will ever be scored as a feature must exist by **6 September, 11:00 PM IST**. The 8–11 September window is explicitly limited to refining demos, pitches, and UI. Plan the build to be feature-complete on 3 September so 4–6 September is genuinely available for submission assets, which always take longer than teams expect.

3 Current State Audit

3.1 Module inventory

Total repository content is 1,433 lines, of which **605 are active code** — roughly 58% of the file content is commented-out prior versions. That dead weight should be stripped during the migration; it obscures the working code and inflates apparent complexity for anyone reviewing the repo.

MODULE	TOTAL	ACTIVE	RESPONSIBILITY
<code>whisper/audio_recorder.py</code>	117	78	Mic capture, noise-floor calibration, record-until-silence
<code>whisper/whisper_asr.py</code>	77	48	Local Whisper <code>medium</code> , device/precision selection, transcription
<code>llama/question_generator.py</code>	284	77	Ollama calls for main questions and follow-ups
<code>evaluation/rubric_generator.py</code>	82	60	LLM-generated concept rubric, weights normalised to 10
<code>evaluation/answer_evaluator.py</code>	181	104	Concept detection, scoring, dodge penalty, LLM feedback
<code>evaluation/topic_aggregator.py</code>	51	37	Per-topic averaging, strengths and weaknesses
<code>controller/interview_controller.py</code>	433	85	Plan-driven state machine, rubric store, history log
<code>pipeline/test_flow.py</code>	205	116	Blocking console orchestration loop
Total	1,433	605	

3.2 Capability gap matrix

PS11 CAPABILITY	STATUS	ASSESSMENT
Agora as core	Absent	No Agora code anywhere in the repository. Disqualifying on its own.
Real-time, interruptible voice	Absent	The pipeline is strictly half-duplex: the interviewer "speaks" (actually <code>print()</code>), then <code>record_audio_until_silence()</code> blocks. Nothing listens while the AI talks, so barge-in is architecturally impossible. There is also no TTS at all.
Multiple interviewer roles	Absent	One hardcoded persona: "You are a strict technical interviewer."
Shared context across roles	Partial	<code>controller.history</code> is a sound foundation but is single-role and is never fed back into question generation.
Dynamic follow-ups	Partial	<code>generate_followup</code> exists, but the <i>decision</i> to follow up is hardcoded — exactly one after every main question. The evaluation result never influences it. That is a fixed pattern, not adaptivity.
Controlled turn-taking	Absent	Nothing to arbitrate; there is one speaker.
Role-play scenarios	Absent	Not attempted.
Difficulty adjustment	Absent	The most misleading gap. <code>evaluation["score"]</code> is computed and logged, then never read. <code>self.plan</code> is fixed at construction. A candidate scoring 2/10 gets the identical ladder as one scoring 10/10.
Vague / contradictory answers	Partial	<code>PUNTING_PHRASES</code> is a crude vagueness proxy. Contradiction detection is entirely absent — it needs cross-turn claim tracking, which does not exist.
Evidence-linked feedback	Absent	Feedback prose exists, but with no turn IDs, timestamps, or quoted spans. Nothing is linked to anything.
Structured final assessment	Broken	A flat print loop. <code>TopicAggregator</code> is never imported anywhere, and is defective regardless (§3.3).
AI disclosure	Absent	Not attempted.

3.3 Defects found in the existing code

These matter because three of them sit in code paths scheduled for reuse. Fix the underlying concepts during the port rather than carrying them across.

LOCATION	DEFECT AND CONSEQUENCE
<code>topic_aggregator.py:24</code>	<code>summarize</code> is defined at module scope, outside the class (zero indentation). <code>TopicAggregator().summarize()</code> raises <code>AttributeError</code> . The final-assessment module has never successfully run.
<code>rubric_generator.py:67</code>	<code>json.loads(raw)</code> is called directly on raw model output , with no brace extraction — unlike <code>answer_evaluator</code> , which does <code>raw.find("{")</code> . Llama 3 routinely prefixes JSON with prose. When it does, the exception is swallowed and the two-concept fallback rubric is returned silently.
<code>answer_evaluator.py:36–40</code>	Concept matching succeeds if any single word of a concept appears anywhere in the answer. With multi-word concepts containing common words, nearly every concept registers as detected and scores inflate toward the 10-point ceiling.
<code>test_flow.py:135</code>	Follow-ups reuse the main question's rubric , so follow-up answers are graded against criteria written for a different question.
<code>test_flow.py:109–119</code>	On question or rubric failure the loop calls <code>continue</code> , but <code>next_step()</code> has already set <code>expect_followup = True</code> . The retry therefore returns <code>ASK_FOLLOWUP</code> with <code>last_question = None</code> , desyncing the plan and generating a follow-up to nothing.
<code>question_generator.py:197–198</code>	<code>text.split(":")[-1]</code> truncates any generated question containing a colon. " <i>Consider this scenario: how would you...</i> " silently becomes " <i>how would you...</i> ".

3.4 The compounding failure — verify this before relying on the evaluation engine

TWO DEFECTS CHAIN INTO A SILENT TOTAL FAILURE

Defect 2 causes `generate_rubric()` to fall back to `[{"concept": "core concept", "weight": 5}, {"concept": "example", "weight": 5}]` whenever the model wraps its JSON in prose — with no error printed, because the `except` is bare.

That fallback then meets defect 3. The words "*core*", "*concept*" and "*example*" match almost any English prose under any-single-word matching. The result is a near-10/10 score on every answer regardless of quality, with nothing in the logs to indicate anything went wrong.

Two-minute check: call `generate_rubric("What is database indexing?")` and inspect the return. Five to seven substantive concepts means the engine works. Two generic concepts means it has never actually run, and the observed scores are meaningless.

This does not change the reuse plan — the *design* of rubric-anchored scoring remains the right one and remains the team's genuine head start. It does mean the sprint must budget validation time rather than assuming the module is proven.

4 Target Architecture

4.1 Why the voice stack is retired rather than adapted

Agora Conversational AI Engine is not a TTS vendor bolted onto your loop; it is the entire real-time conversational pipeline — microphone transport, voice activity detection, speech recognition, barge-in handling, turn detection, LLM invocation, and speech synthesis. Keeping Whisper alongside it produces three specific problems:

- **Duplication reads as bolt-on.** "Use of Agora Conversational AI" is a scored criterion. A parallel local ASR path invites the judgement that Agora is decorative.
- **Interruption cannot be retrofitted.** Barge-in requires continuously processing candidate audio *while* the agent is speaking. `record_audio_until_silence()` is a blocking call by construction — there is no version of it that supports interruption.
- **Latency is scored.** Whisper `medium` is a batch model; combined with a 2.5-second silence threshold before transcription even begins, the floor on response time is several seconds. Agora's streaming pipeline targets sub-second.

Similarly, **local Ollama cannot serve the demo path.** Agora invokes your LLM endpoint over the public internet, so it must be a reachable HTTPS service. A laptop-bound `localhost:11434` is unreachable by definition, and an 8B model on consumer hardware would undermine the conversational quality score regardless.

4.2 The bridge — an OpenAI-compatible gateway

Agora's agent configuration accepts an arbitrary LLM endpoint, declared with `"style": "openai"`, that must implement OpenAI's `/chat/completions` contract. This single fact is what makes the migration tractable: rather than rewriting your interview logic to fit a vendor SDK, you expose it as a chat-completions endpoint and Agora drives it as if it were a model.

THE FREE STACK MAKES THIS SIMPLER, NOT HARDER

Groq, Cerebras, OpenRouter and Gemini all expose OpenAI-compatible chat-completions endpoints. Agora speaks OpenAI; your provider speaks OpenAI; the gateway sits between them injecting conductor logic and streaming frames straight through. The message-translation module that a non-OpenAI provider would require simply does not exist in this design — roughly 100 lines of work removed, and an entire class of format bugs with it.

```

# Agora → your gateway (Agora is the client)
POST /v1/{session_id}/{role}/chat/completions
{ "model": "...", "messages": [...], "stream": true }

# your gateway → Agora (OpenAI-shaped SSE frames)
data: {"choices":[{"delta":{"content":"So walk me through"},"index":0}]}
data: {"choices":[{"delta":{"content":" how you'd shard that."},"index":0}]}
data: [DONE]

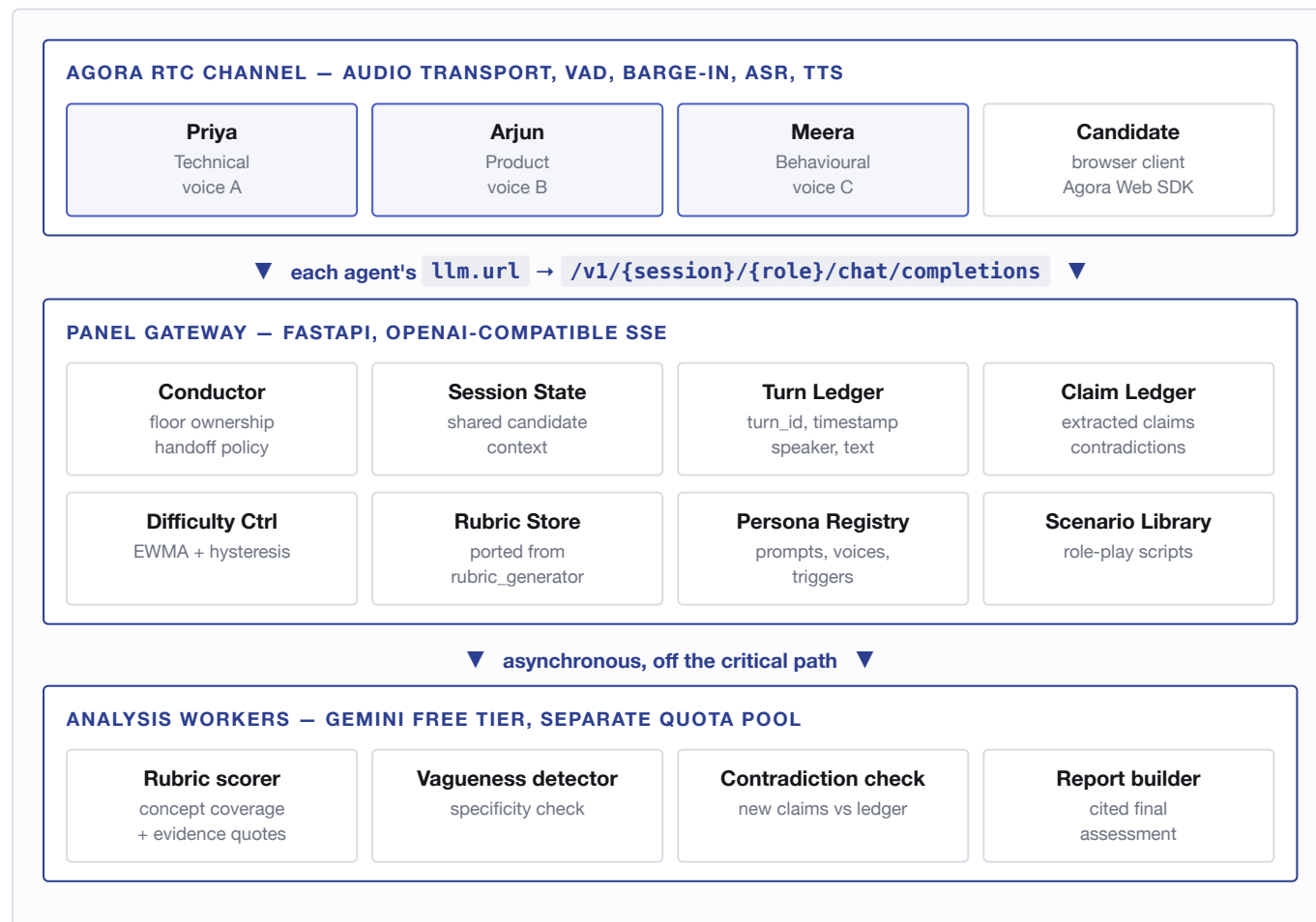
# inside the gateway – Groq via the OpenAI SDK, no translation needed
client = OpenAI(
    base_url="https://api.groq.com/openai/v1",
    api_key=os.environ["GROQ_API_KEY"],
)

if not conductor.holds_floor(session, role):
    yield "data: [DONE]\n\n" # silent persona – no inference, no cost
    return

stream = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=persona_system(role) + conductor.context(session) + msgs,
    tools=CONDUCTOR_TOOLS,
    stream=True, temperature=0.6, max_tokens=300,
)
for chunk in stream:
    yield passthrough(chunk) # already OpenAI-shaped

```

4.3 System diagram



4.4 Floor control — how the panel takes turns

Running one agent per persona is what makes the panel *sound* like a panel: each gets its own TTS voice, and voice distinctness is what sells the concept in a live demo far more than prompt engineering does. The cost of that choice is that all three agents are subscribed to the candidate, so all three call the gateway when the candidate stops speaking.

The Conductor resolves this. It holds a single `floor_holder` per turn:

- **Floor holder** — the gateway streams a real, tool-enabled response.
- **Everyone else** — the gateway returns an immediate empty completion followed by `[DONE]` . No text, no speech, negligible cost, no inference performed.

The advantages are that turn-taking policy lives entirely in code you can show and explain, it relies on no undocumented platform behaviour, and it degrades safely — a bug produces silence, never three agents talking over each other.

PLAN B — VALIDATE THE EMPTY-COMPLETION PATH ON DAY ONE

Confirm during Phase 0 that an empty completion produces genuine silence rather than a TTS artefact or an error state. If it does not, the fallback is to join non-active agents with `remote_rtc_uids: []` so they subscribe to nobody, and swap subscriptions through the `update` endpoint on handoff. That path is more moving parts and adds handoff latency, which is why it is the fallback — but knowing on 25 August rather than 2 September is the difference between an inconvenience and a crisis.

Handoff policy

TRIGGER	CONDUCTOR ACTION
Persona calls <code>request_handoff</code>	Transfer floor to the named role immediately; inject the stated reason into that persona's context.
Contradiction flagged	Transfer floor to the persona who asked the original question — they have the strongest claim to challenge it.
Technical answer accepted with no business framing	Transfer to Product. <i>This is the PS11 example scenario, encoded as policy.</i>
Same persona has held the floor for 3 consecutive turns	Rotate to the next persona by round-robin, to keep all three visibly active.
Two consecutive vague answers on one topic	Current persona retains the floor and switches to a pin-down follow-up.
Scenario active	Floor is locked to the scenario owner until the scenario completes or is abandoned.

4.5 Conductor tools — making tool calling do real work

The hackathon explicitly highlights "AI agents with tool calling." Rather than attaching an unrelated third-party API, expose the Conductor's own controls as tools. This satisfies the tool-calling expectation *and* implements five PS11 capabilities through one mechanism — and it demos well, because tool calls can be surfaced live in the UI as the judges watch.

TOOL	PURPOSE	PS11 LINE
<code>request_handoff</code>	Yield the floor to another role, with a stated reason	5, 2
<code>flag_vague_answer</code>	Record a vague response with the offending quote	8
<code>flag_contradiction</code>	Link two turn IDs that conflict	8
<code>adjust_difficulty</code>	Escalate or de-escalate the current level	7
<code>launch_scenario</code>	Enter a role-play scenario and lock the floor	6
<code>record_evidence</code>	Attach a quote to a competency for the final report	9, 10

Define these with `strict: true` and `additionalProperties: false` so tool inputs validate exactly, and always parse tool inputs with `json.loads()` rather than string matching.

4.6 Model selection and the latency budget

Two paths with opposite requirements. The utterance path is latency-critical and must stream; the analysis path is quality-critical and runs asynchronously where latency is irrelevant. Both run on free tiers — see §5 for quota mathematics and fallback ordering.

PATH	PROVIDER & MODEL	CONFIGURATION AND RATIONALE
Utterance what the persona says	Groq <code>llama-3.3-70b-versatile</code> <i>free tier</i>	Groq's inference speed is the reason to choose it — hundreds of tokens per second, which serves the "quality of conversational experience" criterion directly. OpenAI-compatible at <code>api.groq.com/openai/v1</code> , so the gateway passes frames straight through. Stream always; cap <code>max_tokens</code> around 300 because interviewers should be terse, and short replies also stretch the token quota.
Analysis scoring, contradictions, report	Gemini <code>2.5 Flash</code> / <code>2.5 Flash-Lite</code> <i>free tier</i>	Deliberately a <i>different provider</i> so analysis load draws on a separate quota pool and can never starve the voice path. Use <code>response_schema</code> for structured JSON. Runs off the critical path, so latency is irrelevant — batch several turns' analysis into one call if daily request limits get tight.

KEEP THE JSON-REPAIR PLUMBING AFTER ALL

On a paid Claude stack, schema-guaranteed structured outputs would let you delete the brace-extraction and retry code in `answer_evaluator.llm_feedback()`. Free-tier Llama and Gemini JSON modes are good but not schema-guaranteed, so **that defensive code is an asset again, not dead weight**. Port it, keep the retry loop, and keep a safe fallback object — but make the fallback **log loudly** rather than failing silently, which is what causes the compounding failure in §3.4.

Cost and quota discipline replaces prompt caching

Free tiers meter requests and tokens, not dollars, so the optimisation target changes. There is no cross-request prompt cache to exploit; instead, minimise tokens per call and calls per turn:

- **Silent personas cost nothing.** Non-floor agents return `[DONE]` before any provider call. With three personas this is a 3× saving on every turn — the single biggest quota lever in the design.
- **Cap the history window.** Send the last 8–10 turns plus a running summary, not the full transcript. Set Agora's `max_history` to match.
- **Batch the analysis path.** Score two or three turns in one request when daily limits tighten.
- **Keep persona prompts short.** Every token is resent on every turn; a 900-token persona prompt across 25 turns is 22K tokens against a per-minute ceiling.

- **Log token usage per turn** from day one, so you learn your real per-interview cost before demo day rather than during it.

Latency budget

STAGE	TARGET	NOTES
Agora VAD + ASR finalisation	~300 ms	Platform-controlled; tune VAD silence duration.
Gateway routing + conductor decision	< 20 ms	Pure in-memory logic. No LLM call.
LLM time-to-first-token (Groq)	< 400 ms	Groq's speed is why it is the utterance provider. Measure it; do not assume.
TTS first audio byte	~200 ms	Use an Agora-managed voice — see §5.2 on why bringing your own key costs more.
End of speech → first audio	< 1.0 s	Instrument this from day one and put the number on a slide.
Analysis workers	async	Never on the critical path. Results inform the <i>next</i> turn.

4.7 Data model

Three ledgers underpin every remaining PS11 capability. Build them in Phase 2, before the features that depend on them — retrofitting turn IDs into an existing transcript is painful and error-prone.

Turn	turn_id, session_id, speaker(role candidate), text, started_at, ended_at, difficulty_at_time
Claim	claim_id, turn_id, text, topic, verbatim_quote, contradicts[] → claim_id
Evidence	evidence_id, turn_id, competency, quote, polarity, rubric_concept, weight
SessionState	session_id, candidate_ctx, floor_holder, difficulty, rolling_score, personas[], active_scenario, turns[], claims[], evidence[]

Every sentence in the final report must resolve to at least one Evidence row, and every Evidence row carries the turn_id and quote that justify it. That is what "evidence-based feedback linked to the interview transcript" means in practice, and it is straightforward once the ledgers exist.

5 Zero-Budget Stack & Quota Discipline

Every component of this architecture has a viable free tier except one. Agora is mandated by the rules and is metered per agent-minute, which makes agent-minutes — not dollars, not tokens — the scarce resource the whole project must be engineered around.

5.1 What each component costs

COMPONENT	CHOICE	FREE ALLOWANCE	WHAT HAPPENS AT THE LIMIT
Real-time voice	Agora Conversational AI	300 agent-minutes / month , then \$0.10 per agent-minute	Hard cost The only unavoidable spend. Mandated by the rules. See §5.2.
ASR	Agora-managed	Included in the agent-minute price	No separate bill. Do not bring your own key — see §5.2.
TTS	Agora-managed voices	Included in the agent-minute price	No separate bill, and you still get multiple distinct voices for the panel.
LLM — utterance	Groq	30 RPM; 1,000 req/day on <code>llama-3.3-70b-versatile</code> ; ~12K TPM. No card required.	HTTP 429. Fall back to the next provider in the chain (§5.3).
LLM — analysis	Gemini	~250 req/day (2.5 Flash), ~1,000 (Flash-Lite); 250K TPM	Separate quota pool, so it cannot starve the voice path. Batch on pressure.
Gateway hosting	Hugging Face Spaces (Docker + FastAPI)	Free, public HTTPS, no forced sleep on CPU	Judge-facing persistent URL. Cloudflare Tunnel to a laptop for live demos.
Web client	Vercel / Netlify / GitHub Pages	Free static hosting	Ample for a single-page Agora Web SDK client.
State & ledgers	SQLite or in-process + JSON export	Free	An interview session is small. No managed database is needed.
Report PDF	WeasyPrint or ReportLab	Free, local	No document API required.

5.2 The agent-minute budget — the number that governs the project

DO THE MULTIPLICATION BEFORE YOU DESIGN THE PANEL

Agora bills per **agent-minute**, not per session-minute. Three personas in a ten-minute interview consume **30 agent-minutes**, not ten.

That means the 300-minute monthly free allowance is **ten full three-persona interviews** — for the entire month, covering development, integration testing, rehearsal, the demo video, and the live demo. Careless testing will exhaust it in an afternoon.

CONFIGURATION	AGENT-MIN / SESSION	SESSIONS IN 300 MIN	VERDICT
3 personas × 10 min	30	10	Full design. Viable only with strict discipline.
3 personas × 4 min (test)	12	25	Use short scripted sessions for integration tests.
2 personas × 10 min	20	15	Recommended fallback. PS11 says "one or more" — two is fully compliant and still demonstrates cross-role challenge.
1 persona × 10 min	10	30	Compliant, but forfeits the differentiator. Emergency only.

Four mitigations, in order of value

1 Ask for hackathon credits on Discord — do this first

The event is *run with Agora*, and sponsors routinely issue participant credits that are not advertised on the public pricing page. A single message on the Agora Discord may remove this entire constraint. Nothing else on this list has a better effort-to-payoff ratio.

2 Build a mock harness and develop against it

Write a local script that replays a scripted interview into the gateway using the exact OpenAI request shape Agora sends. Conductor logic, floor control, scoring, contradiction detection, difficulty laddering and the final report — perhaps 80% of the build — can then be developed and tested with **zero agent-minutes consumed**. Only integration checks and rehearsals touch Agora. This is good engineering practice regardless of budget; here it is also the difference between finishing and running out.

3 Make persona count a configuration flag

Ship with `PANEL_SIZE` switchable between two and three. Develop and test on two, rehearse and demo on three. If credits run short in the final week, you drop to two without a code change.

4 Enforce short idle timeouts and guaranteed teardown

Set `idle_timeout` aggressively and call `leave` in a `finally` block. An agent left running after a crashed test session silently bills until it times out — this is the most common way hackathon teams lose credits.

BRINGING YOUR OWN TTS KEY DOES NOT SAVE MONEY

Agora's pricing is explicit that the agent-minute price **covers the bundled ASR, LLM and TTS models, and is charged identically if you bring your own key**. Worse, choosing a vendor outside the managed list — ElevenLabs, for example — means paying Agora's \$0.10 *plus* that vendor's bill.

The correct move is therefore the opposite of the usual free-tier instinct: **use Agora-managed ASR and TTS**, and spend your free-tier effort on the LLM, which is the one component the bundle lets you replace at no extra charge by pointing `llm.url` at your own gateway.

5.3 Provider fallback — rate limits are the demo-day risk

A paid API degrades gracefully under load; a free tier returns HTTP 429 and stops. During a live judged demo that is indistinguishable from a crash. Because every provider below is OpenAI-compatible, a fallback chain costs little to build and removes the single most likely on-stage failure.

ORDER	PROVIDER	ROLE IN THE CHAIN
1 – primary	Groq	Fastest inference; the default for every utterance.
2 – failover	Cerebras	Comparable speed, independent quota. Free tier.
3 – failover	OpenRouter (:free models)	Broad model choice, independent quota. Slower.
4 – analysis only	Gemini	Never on the voice path; keeps its quota reserved for scoring.
5 – last resort	Local Ollama + Cloudflare Tunnel	Unlimited and offline-capable, but laptop-speed. Acceptable as a "never fully dead" backstop, not as a demo path.

- Give every team member their **own API keys** for development, so one person's testing cannot exhaust the quota the others depend on.
- Reserve a **clean, untouched key** used only for the demo video recording and the live demo.
- Catch 429 explicitly and advance the chain — never let a rate limit surface as a stalled agent.
- Log which provider served each turn, so post-demo you can explain any latency variation rather than guessing.

5.4 What the free stack costs you in capability

Being honest about the trade-offs, so nothing is a surprise during the sprint:

TRADE-OFF	CONSEQUENCE AND MITIGATION
No schema-guaranteed structured output	Free-tier JSON modes are good, not guaranteed. Keep the ported retry-and-repair code, and log every fallback loudly (§4.6).
Weaker instruction-following on nuanced persona behaviour	Llama 70B holds a persona well but drifts more than a frontier model. Compensate with shorter, sharper persona prompts and explicit few-shot examples of the desired tone.
No cross-request prompt caching	Optimise for fewer tokens per call instead: capped history window, terse prompts, batched analysis.
Daily request ceilings	The mock harness (§5.2) keeps development off the metered path entirely, which protects both Agora minutes and LLM quotas at once.
Contradiction detection quality	The subtlest capability to get right on a smaller model. Narrow the task: compare each new claim only against claims on the same topic, not the whole ledger, and require the model to quote both sides.

NONE OF THIS WEAKENS THE SUBMISSION

Nothing in the evaluation criteria rewards an expensive model. Judges score innovation, use of Agora, technical implementation, conversational quality, impact, and readiness. A fast free model with a well-designed conductor will out-score a frontier model behind a sloppy one — and Groq's latency advantage arguably makes the conversation *feel* better than a slower, more capable model would.

6 Code Reuse Disposition

6.1 File-by-file verdict

FILE	ACTIVE	VERDICT	CARRIES	WHAT HAPPENS
<code>rubric_generator.py</code>	60	Port	~50	Highest-reuse file. <code>normalize_weights()</code> is a pure function — copy verbatim. Swap the Ollama transport for the Gemini free tier with <code>response_schema</code> , and add the brace-extraction guard the module currently lacks — together these fix defect 2. Extend to per-role rubrics and pre-generate off the critical path.
<code>answer_evaluator.py</code>	104	Split	~45	Keep: <code>compute_score()</code> verbatim; <code>PUNTING_PHRASES</code> + <code>dodge_penalty()</code> as the first pass at vagueness detection; and the JSON-repair block — on free-tier models it earns its place (§4.6), but make its fallback log loudly instead of failing silently. Rewrite: <code>detect_concepts()</code> becomes an LLM judge returning covered concepts <i>plus the verbatim quote evidencing each</i> — fixing the scoring and producing the evidence trail in one change.
<code>topic_aggregator.py</code>	37	Port	~30	Correct shape for the final assessment. Fix the indentation defect, re-key from <code>topic</code> to <code>role</code> / <code>competency</code> , and attach <code>turn_id</code> lists to each strength and weakness so the report can cite.
<code>interview_controller.py</code>	85	Partial	~25	<code>store_rubric</code> , <code>get_rubric</code> , <code>log_interaction</code> , <code>get_history</code> become <code>SessionState</code> nearly verbatim, plus <code>turn_id</code> , <code>timestamp</code> and <code>role</code> . <code>next_step()</code> does not survive — it is pull-based and single-speaker where the Conductor must be event-driven and arbitrate N personas. Delete the ~300 commented-out lines.
<code>question_generator.py</code>	77	Prompts	~15	The prompt discipline — "Return ONLY the question," no preamble, no hints, stay on topic — transfers into persona system prompts. The functions largely dissolve: questions stop being discrete strings that get synthesised, because the persona streams its utterance through the gateway. Drop the colon sanitiser entirely.
<code>test_flow.py</code>	116	Delete	0	No code survives — it is a blocking console loop. But the <i>sequence</i> it encodes (question → rubric → answer → evaluate → log → decide next) is the gateway's flow specification. Keep it open as a checklist during Phase 2, then delete.
<code>whisper_asr.py</code>	48	Delete	0	Replaced wholesale by Agora ASR.
<code>audio_recorder.py</code>	78	Delete	0	Replaced wholesale by Agora VAD and transport.
Total	605		~165	≈ 27% carries forward, concentrated in <code>evaluation/</code> .

THE RETIRED VOICE STACK STILL HAS PITCH VALUE

Do not treat the 126 deleted lines as wasted. "We built a local Whisper and VAD pipeline, measured its end-to-end latency, and moved to Agora for a measured improvement" is a strong architecture slide with a real number attached — and it directly evidences the "Use of Agora Conversational AI" criterion by showing you understand what the platform provides. Preserve the code on the `pre-hackathon-baseline` tag and benchmark it once before deleting.

6.2 Target repository structure

Stay in the same repository. Git history is the evidence for the mandatory prior-work disclosure; a fresh repo discards it.

```
src/
├── agora/
│   ├── session.py           # channel creation, RTC token minting
│   ├── agent_lifecycle.py   # join / leave / update / speak / interrupt
│   └── personas.py          # role → voice, prompt, triggers
├── gateway/
│   ├── app.py               # FastAPI, OpenAI-compatible routes
│   ├── sse.py               # OpenAI SSE frame encoding
│   └── providers.py          # Groq → Cerebras → OpenRouter fallback chain
├── conductor/
│   ├── conductor.py         # floor ownership, handoff policy
│   ├── tools.py             # the six conductor tools
│   ├── difficulty.py        # EWMA ladder with hysteresis
│   └── scenarios.py          # role-play library
├── state/
│   ├── session_state.py     ← ported from interview_controller
│   ├── turn_ledger.py       # turn_id, timestamps, speaker
│   └── claim_ledger.py       # claims + contradiction links
├── analysis/
│   ├── rubrics.py           ← ported from rubric_generator
│   ├── scorer.py            ← ported from answer_evaluator
│   ├── contradiction.py     # new
│   └── report.py            ← ported from topic_aggregator
├── web/                      # candidate client, Agora Web SDK
├── mock/
│   └── replay.py            # scripted interview → gateway, 0 agent-minutes
```

6.3 Migration sequence

1 Tag the baseline first

`git tag pre-hackathon-baseline` before touching anything. This is the artefact the disclosure points at.

2 Benchmark, then delete the voice stack

Record one end-to-end latency measurement for the slide, then `git rm` the `whisper/` package and `test_flow.py`.

3 Strip dead code

Remove the ~800 commented-out lines. The repo should read as ~600 lines of live code, not 1,400 of archaeology.

4 Port `evaluation/` into `analysis/`

Swap the Ollama transport for the Gemini free tier, fix the three defects, keep the JSON-repair path but make it log, and add evidence quotes to the scorer output.

5 Build outward from the gateway

Gateway first (it is the contract), then conductor, then agora lifecycle, then web client.

7 The Roadmap

Phase 0 decides whether the rest is comfortable or frantic — it exists to prove the platform works before the sprint clock starts, and it must not be skipped in favour of writing interview logic. The sprint itself is then broken into six named days, each with a single goal and a testable acceptance condition.

7.1 Three lanes, three owners

Split by system boundary rather than by feature, so the three lanes touch different files and rarely block each other. Every task in this roadmap is tagged with its lane.

LANE	OWNS	RESPONSIBILITY
DEV A Platform	agora/ , gateway/ , mock/	Agent lifecycle, tokens, the OpenAI-compatible endpoint, SSE streaming, provider fallback, deployment, the mock harness.
DEV B Intelligence	conductor/ , state/ , analysis/	Floor control, handoff policy, ledgers, rubrics, scoring, contradiction detection, difficulty, the final report.
DEV C Experience	web/ , assets	Candidate client, observer view, transcript UI, demo video, pitch deck.

If the team is two people: A takes platform plus the web client; B takes intelligence plus the report; both share the video and deck on 4–5 September. Drop the observer view (Day 6) first if time runs short — it is the only genuinely optional item in the sprint.

THE INTEGRATION CONTRACT — AGREE THIS ON DAY 1 AND DO NOT CHANGE IT

Lane A and Lane B meet at exactly one function signature. Fix it before either starts writing, and both can work without blocking:

```
def next_utterance(session_id, role, messages) -> Iterator[str] | None
    # returns None → this persona is silent this turn (gateway emits [DONE])
    # returns iter → this persona holds the floor; gateway streams the chunks
```

Lane A implements the HTTP, SSE and provider plumbing around it. Lane B implements everything behind it. The mock harness calls the same function, so Lane B never needs Agora running to work.

The single highest-value activity this week is proving the Agora path end to end. Writing interview logic now risks building on an assumption you have not tested.

1 Register the team on Commudle

Mandatory. Minimum two members, maximum four. Team lead joins the WhatsApp group; all members join Discord. *Registration closes 25 August.*

2 Provision Agora and ask for hackathon credits

Create the account and project, obtain App ID and App Certificate, obtain Customer ID and Customer Secret for REST authentication, and enable Conversational AI Engine. **Then post on the Agora Discord asking whether participant credits are available beyond the 300 free agent-minutes.** This is the highest-payoff message anyone on the team will send all month (§5.2).

3 Lock three Agora-managed voices

Do not defer this, and do not bring your own TTS key — it costs the same or more (§5.2). Pick three managed voices that differ in pitch and pace, not just accent, and measure first-audio-byte latency before committing.

4 Get free LLM keys and stand up the gateway

Groq for the utterance path, Gemini for analysis, plus one failover key. Deploy a public HTTPS endpoint that accepts the OpenAI request shape, calls Groq, and streams frames back — Hugging Face Spaces for the persistent URL, Cloudflare Tunnel for local iteration.

5 Write the mock replay harness before anything else

A script that posts a scripted interview into the gateway in Agora's exact request shape. Everything in Phases 2–4 is then developed against it at zero agent-minute cost. Building this on day one, rather than after credits run low, is what keeps the sprint solvent (§5.2).

6 Run the end-to-end smoke test

Browser joins the channel → REST `join` starts one agent pointed at your endpoint → hold a spoken conversation → **interrupt the agent mid-sentence and confirm it yields.**

7 Validate the silent-agent path

Return an empty completion and confirm it produces genuine silence, not an error or artefact. This decides whether §4.4's primary design or its Plan B is the one you build.

8 Tag the baseline and write the disclosure

`git tag pre-hackathon-baseline`, then add a disclosure section to the README naming what pre-existed and what will be built during the sprint.

HARD GATE If the smoke test and the silent-agent check are not green by 25 August, escalate on the Agora Discord immediately. Every subsequent phase depends on them, and there is no slack in the sprint to absorb a platform surprise. Keep total Phase 0 spend under 30 agent-minutes — use short sessions.

Goal: get shortlisted. Nothing built here ships, but nothing ships at all if this fails.

A presentation template will be provided. Required sections: problem statement and proposed solution, technical architecture, expected impact, and Agora technologies integration. Top 250–500 teams advance.

1 Name the product

"AI-Interview" is a description, not a name. Pick something that survives a pitch.

2 Lead with the panel

Most PS11 submissions will be single-interviewer bots. A conducted multi-persona panel with distinct voices is the differentiator — put it in the first two slides, not the appendix.

3 Use the §4.3 diagram and name the endpoints

Judges screening hundreds of decks reward specificity. Name `join`, `speak`, `interrupt`, `update`, and `history` explicitly rather than writing "we will use Agora."

4 Frame impact commercially

Panel time is the scarcest resource in hiring. An adaptive panel replaces screening rounds rather than replacing quizzes — that framing carries into the Round 5 business-potential criterion.

5 Rework the existing deck

Build on `AI_Interview_Presentation.pptx` rather than starting from a blank file.

Goal: one persona, one voice, one real spoken exchange through your own gateway. Nothing clever — just the whole pipe connected.

1 Book the 1:1 mentorship session ALL

The guidelines give each team one dedicated technical session in Round 3. Book it now for Day 3 or 4 — early enough to act on, late enough that you have real questions. Prepare three specific Agora questions in advance: silent-agent behaviour, participant credits, and multi-agent-in-one-channel patterns. Do not spend it on architecture you can decide yourselves.

2 Repo surgery ALL

Tag, delete, strip. Thirty minutes, done once, and it stops everyone tripping over dead code all week.

```
git tag pre-hackathon-baseline && git push --tags
git rm -r src/back_end/whisper src/back_end/pipeline
# strip ~800 commented-out lines from controller + question_generator
# create the package skeleton from §6.2
```

3 Persona registry as data DEV B

One YAML or JSON file: role key, display name, system prompt, Agora voice ID, question domains, handoff triggers. Data not code, so personas are tunable without redeploy — you will tune them a dozen times this week.

4 Gateway skeleton DEV A

FastAPI route `/v1/{session}/{role}/chat/completions`. Verify the shared secret, parse the OpenAI request, call `next_utterance()`, stream SSE frames, terminate with `[DONE]`. Get the framing exactly right today — every latency number downstream depends on it.

5 Provider fallback chain DEV A

Groq primary; catch 429 and advance to Cerebras, then OpenRouter. Log which provider served each turn.

6 Mock replay harness DEV A

A script that posts a scripted 10-turn interview into the gateway in Agora's exact request shape. **From this point Lane B never needs Agora running** — which is what keeps the agent-minute budget intact.

7 Session service and first join DEV A

Mint RTC tokens, join *one* agent, hold a spoken exchange, leave cleanly. Wrap teardown in `finally` so a crash cannot leave an agent billing.

ACCEPTANCE You speak to one Agora agent, it answers using text your gateway produced, and the mock harness drives the same code path with zero agent-minutes.

Goal: three voices in one channel, exactly one audible per turn, every word recorded with an ID and a timestamp.

1 **Turn ledger first** DEV B

Before any conductor logic. Every utterance from either side gets `turn_id`, speaker, text, start and end timestamps, and the difficulty level at the time. Retrofitting IDs later is painful and the report depends on them.

2 **Session state** DEV B

Port `store_rubric` / `get_rubric` / `log_interaction` / `get_history` from `interview_controller` almost verbatim, adding role and timestamp.

3 **Conductor v1 — floor control** DEV B

A single `floor_holder` per turn. Floor holder streams; every other persona returns `None` so the gateway emits `[DONE]` with no inference. Round-robin only for now — real policy lands tomorrow.

4 **Multi-agent join** DEV A

Join `PANEL_SIZE` agents, each with its own UID, voice and `llm.url` role segment. Make `PANEL_SIZE` an env var today, not later — it is your credit-pressure release valve.

5 **AI disclosure** DEV A

Set the lead persona's `greeting_message` to state plainly that all interviewers are AI. One config line, one complete PS11 capability.

6 **Candidate client v1** DEV C

Agora Web SDK: join channel, publish mic, subscribe to agents, render live transcript. Ugly is fine today.

ACCEPTANCE Three distinct voices; exactly one speaks per turn; you can interrupt any of them mid-sentence and it yields; every utterance lands in the ledger with a timestamp. Use 4-minute test sessions — 12 agent-minutes each.

Goal: personas hand the floor to each other for stated reasons — including the PS11 example scenario.

1 **The six conductor tools** DEV B

Define `request_handoff`, `flag_vague_answer`, `flag_contradiction`, `adjust_difficulty`, `launch_scenario`, `record_evidence` with strict schemas. Parse inputs with `json.loads()`, never string matching.

2 **Tool-call interception in the stream** DEV A

Tool calls arrive interleaved with text in the SSE stream. Strip them out, dispatch to the conductor, and forward only spoken words to Agora — a leaked tool call would be read aloud as gibberish.

3 **Handoff policy** DEV B

Implement the §4.4 table: explicit handoff, contradiction routes to the original asker, three-turn rotation cap, scenario floor lock, and the technical-accepts-but-product-challenges rule the demo depends on.

4 **Shared context injection** DEV B

Each persona's prompt receives a compact digest of what the candidate told the *others* — the mechanism behind PS11 capability 3. Cap it at 8–10 turns plus a running summary.

5 **Speaking indicator** DEV C

Panel avatars with an unmistakable "speaking now" state. This is what makes turn-taking legible to a judge watching a video.

ACCEPTANCE A technically sound answer with no business framing causes Priya to accept and Arjun to challenge, in different voices, referencing what the candidate told Priya.

Goal: the interview visibly responds to how well the candidate is doing.

1 **Run the §3.4 check before porting anything** DEV B

Call `generate_rubric("What is database indexing?")`. Five to seven substantive concepts means the engine works; two generic ones means it never has. Two minutes, and it determines whether the next task is a port or a rewrite.

2 **Port rubrics to Gemini** DEV B

Keep `normalize_weights()` verbatim. Swap the transport, add `response_schema`, add the brace-extraction guard the module currently lacks, and make the fallback **log loudly**. Pre-generate per role, never inside the response path.

3 **Async analysis worker** DEV B

Fire-and-forget after each candidate turn: rubric score with evidence quotes, vagueness check, claim extraction. Results inform the *next* turn and must never block the current one.

4 **Evidence-producing scorer** DEV B

Rewrite `detect_concepts()` as an LLM judge returning, per concept, whether it was covered *and the verbatim quote proving it*. This fixes the broken matching and produces the citation trail in one change.

5 **Difficulty controller** DEV B

EWMA over answer scores with hysteresis — two consecutive turns outside the band before shifting, so the level does not oscillate.

```
ewma = 0.4 * score + 0.6 * ewma
band: <4 easy | 4-7 medium | >7 hard
shift only after 2 consecutive out-of-band turns
expose level → persona prompt + UI meter
```

6 **Difficulty meter in the UI** DEV C

A live indicator. An adaptation nobody can see scores nothing.

ACCEPTANCE Two weak answers visibly drop the difficulty level; every score carries at least one quoted span from the transcript. All testable through the mock harness — no agent-minutes needed.

Goal: the two hardest PS11 capabilities, plus the artefact that turns a demo into a product.

1 **Claim ledger** DEV B

Extract claims per turn as `{claim, topic, turn_id, quote}`. Keep it narrow — a claim is an assertion about how something works or what the candidate did.

2 **Contradiction detection** DEV B

The subtlest task on a free-tier model, so constrain it hard: compare each new claim only against prior claims *on the same topic*, never the whole ledger, and require the model to quote both sides. On a hit, call `flag_contradiction` and route the floor to the original asker.

3 **Scenario library** DEV B

Three or four role-play scenarios per persona, entered via `launch_scenario`, floor locked to the scenario owner until it completes.

4 **Structured final assessment** DEV B

Fix `topic_aggregator`'s indentation defect, re-key from topic to competency, attach `turn_id` lists. Per-competency scores, strengths, gaps, hire/no-hire signal — **every claim citing a turn ID, timestamp and verbatim quote**. Render to PDF with WeasyPrint.

5 **Report view + tool-call feed** DEV C

Render the report in-browser, and surface conductor tool calls live as they fire. Judges cannot score reasoning they cannot see; this is the cheapest way to make it visible.

ACCEPTANCE A deliberately self-contradicting answer is caught and challenged with both quotes shown; the end-of-session report contains no uncited claim.

Goal: three additions that map onto stated hackathon interests, a measured latency number, then stop building.

1 **Multilingual answering** DEV A

Explicitly listed under "What You'll Build". Let the candidate answer in Hindi and have the panel continue seamlessly — a config change plus a UI toggle, not a workstream, and a strong differentiator for an Indian hiring context. Set the ASR language or enable multilingual recognition, pick TTS voices with the matching locale, and instruct personas to mirror the candidate's language.

2 **Human-in-the-loop observer view** DEV C

Also explicitly listed, and it maps straight onto the conductor you already have. A recruiter view showing the live transcript, current floor holder, difficulty level and flags — with three buttons that inject conductor directives: *push harder*, *move on*, *hand to Product*. Roughly half a day on top of existing machinery, and it speaks directly to the product-readiness criterion. **Cut this first if the day is tight.**

3 **Latency instrumentation** DEV A

Measure end-of-speech to first audio byte across ten turns. Pull Agora's per-turn latency metrics from the turn-details endpoint. Put the median and p90 on a slide — very few teams will have a real number.

4 **Persona tuning pass** DEV B

The highest-value hour of the week. Run three full interviews and tighten the prompts until each persona sounds distinct and stays terse. On free-tier models this matters more than any architectural change.

5 **Deploy and test cold** DEV A

Push to Hugging Face Spaces, then open the link on a device that has never run the project, on a network that is not yours.

FEATURE FREEZE — END OF 3 SEPTEMBER. Everything scored as a feature must exist now. Anything unfinished gets cut, not rushed — a smaller demo that runs beats a larger one that fails on stage.

1 **Deploy publicly and test from a clean device**

Judges must be able to open a link. Test from a machine that has never run the project, on a network that is not yours.

2 **Record the demo video**

Follow the §8.1 script, using the reserved clean API keys. Every PS11 capability should be visible on screen, not merely narrated. Budget the agent-minutes for two or three takes.

3 **Write the README**

Architecture, stack, setup instructions, a dedicated top-level "How Agora Conversational AI is integrated" section, and the prior-work disclosure.

4 **Finalise the pitch deck and rehearse the online presentation**

Round 4 includes a live presentation, not just an upload.

5 **Submit with a day in hand**

Target 5 September. Deadline-day submissions lose to upload failures and time-zone arithmetic every year.

1 Build the business case

Round 5 judges on scalability, business potential, real-world impact, and demo quality — not on code. Prepare cost per interview, panel-hours saved, and concurrency ceiling.

2 Rehearse on a hostile network

Conference Wi-Fi is adversarial to real-time audio. Test on a phone hotspot and on a deliberately throttled connection.

3 Record a full-quality fallback video

If the live demo fails on stage, you play the recording and keep talking. Teams that lack this lose the round to circumstance.

4 Prepare for Q&A

Expect: how do you prevent hallucinated scoring; what stops the panel talking over each other; how does this scale to a thousand concurrent interviews; what is your latency. Have specific answers with numbers.

8 Demo Script & Submission Checklist

8.1 The demo narrative

Structure the video and the live demo identically, so the live run is a rehearsal you have already performed dozens of times. Each beat maps to a specific PS11 capability, or — for the last two additions — to a capability the organisers named under "*What You'll Build*".

TIME	BEAT	WHAT IS DEMONSTRATED	PS11
0:00	Session opens; disclosure spoken and shown	The panel introduces itself and states plainly that all three interviewers are AI.	11
0:20	Priya (Technical) opens with a scoped question	Distinct voice; question grounded in the declared role.	2
0:45	Candidate answers; Priya follows up on a specific phrase	The follow-up quotes the answer — visibly derived, not scripted.	4
1:15	Candidate interrupts mid-sentence	Priya stops immediately and yields. Do this deliberately and obviously.	1
1:35	Strong technical answer with no business framing	Priya accepts it, then <code>request_handoff</code> fires on screen.	5
1:50	Arjun (Product) challenges the business impact	Different voice, different concern, referencing what the candidate told Priya. <i>The PS11 example scenario, reproduced exactly.</i>	2,3,5
2:20	Candidate contradicts an earlier claim	<code>flag_contradiction</code> fires; both turn IDs surface in the UI; the original asker challenges it.	8
2:45	Two weak answers; difficulty meter drops	Visible, on-screen adaptation to performance.	7
3:00	Meera launches a behavioural role-play scenario	In-character across multiple turns; floor locked to the scenario owner.	6
3:20	Candidate switches to Hindi mid-answer	The panel continues seamlessly in the same language. Ten seconds of video, and it hits the hackathon's stated interest in multilingual voice experiences.	—
3:35	Cut to the recruiter observer view	A human clicks <i>push harder</i> ; the next question visibly sharpens. Demonstrates human-in-the-loop, also explicitly listed by the organisers.	—
3:50	Session ends; report renders	Per-competency scores with every claim citing a timestamped quote.	9,10
4:10	Latency and architecture slide	Measured end-of-speech-to-audio median and p90, plus the Agora integration diagram.	—

8.2 Round 4 submission checklist

- Project name and short description
- Selected problem statement — **PS11**
- Team member details (2–4 members)
- Public GitHub repository
- Working deployment link, tested from a clean device
- Demo video following §7.1
- Architecture and technology-stack document
- "**How Agora Conversational AI is integrated**" as a top-level README section
- Pitch deck

- Prior-work disclosure naming `pre-hackathon-baseline`
- Licence compliance for every third-party model, dataset and library
- Online presentation rehearsed

INCOMPLETE SUBMISSIONS ARE NOT EVALUATED

The guidelines state plainly that "*incomplete or inaccessible submissions may not be considered for evaluation.*" A private repository, an expired deployment, or an unlisted video is a scoring event, not an administrative detail. Verify every link from a logged-out browser.

9 Risk Register

RISK	IMPACT	EARLY WARNING & MITIGATION
Agora integration slips past 25 Aug	Fatal	Phase 0 is a hard gate specifically to surface this early. Escalate on the Agora Discord on day one rather than day five; mentorship and support exist precisely for this.
Empty completion does not silence an agent	High	Validated in Phase 0 step 6. Fallback: join non-active agents with <code>remote_rtc_uids: []</code> and swap subscriptions via <code>update</code> on handoff.
Agora free minutes exhausted mid-sprint	Fatal	300 agent-minutes is ten three-persona interviews for the whole month (\$5.2). Ask Discord for participant credits on day one; develop against the mock harness so only integration checks and rehearsals are metered; make <code>PANEL_SIZE</code> a flag; enforce <code>idle_timeout</code> and guaranteed teardown so crashed sessions do not bill silently.
LLM free-tier rate limit hit during the live demo	High	A 429 on stage is indistinguishable from a crash. Build the provider fallback chain (§5.3), reserve untouched keys for the demo, give each member their own dev keys, and log which provider served each turn.
Latency stacking makes the panel feel sluggish	High	Non-floor personas return immediately with no inference; all analysis runs asynchronously; prompt caching keeps time-to-first-token low. Instrument from Phase 2, not Phase 4 — you cannot fix what you start measuring on the last day.
Team below the two-member minimum	Fatal	Recruit before 25 August. Solo entries are ineligible regardless of project quality.
Ported evaluation engine turns out never to have worked	Medium	Run the §3.4 two-minute check now, not during the sprint. Structured outputs remove the failure mode, but the rubric prompts still need validation against real questions.
Sunk-cost pull toward the Whisper stack	Medium	Decide now, in writing, that it is retired. Benchmark it once for the architecture slide, then delete it. Revisiting this decision mid-sprint costs days.
Feature creep past the 3 Sep freeze	Medium	No major features may be added after Round 4 evaluation. Anything not working on 3 September should be cut rather than rushed — a smaller demo that runs beats a larger one that fails on stage.
Live demo fails on conference network	High	Rehearse on a hotspot and a throttled connection. Record a full-quality fallback video and be ready to narrate over it without breaking stride.

10 Appendices

Appendix A — Agora Conversational AI REST surface

Base URL: `https://api.agora.io/api/conversational-ai-agent/v2/projects/<appid>` . Request URLs and bodies are case-sensitive. Authenticate with your Customer ID and Customer Secret.

ENDPOINT	PURPOSE IN THIS ARCHITECTURE
POST /join	Start a persona agent in the channel. Carries the full <code>properties</code> object.
POST /agents/{id}/leave	Stop an agent at session end or on teardown.
GET /agents/{id}	Query agent status; used for health checks and orphan cleanup.
POST /agents/{id}/update	Adjust parameters at runtime — token refresh, and subscription swapping if Plan B is needed.
POST /agents/{id}/speak	Force exact text through TTS. Used for the AI disclosure and scripted moderator handoffs.
POST /agents/{id}/interrupt	Hard-cut a speaking agent. Used by the conductor for forced handoffs.
POST /agents/{id}/think	Inject a text instruction into the live conversation pipeline.
GET /agents/{id}/history	Retrieve conversation history after the session.
GET turn details	Per-turn start/end events and latency metrics — the source for the performance slide.

Join request shape

```
{
  "name": "panel-{session}-technical",
  "properties": {
    "channel": "interview-{session}",
    "token": "<rtc token for this agent uid>",
    "agent_rtc_uid": "1001",
    "remote_rtc_uids": ["<candidate uid>"],
    "idle_timeout": 60,
    "asr": { "vendor": "...", "params": { "language": "en-US" } },
    "llm": {
      "style": "openai",
      "url": "https://<your-host>/v1/{session}/technical/chat/completions",
      "api_key": "<shared secret you verify>",
      "system_messages": [ /* persona prompt */ ],
      "greeting_message": "Hi – I'm Priya. Before we start: all three interviewers on this panel are AI.",
      "failure_message": "Give me one moment.",
      "max_history": 32,
      "params": { "model": "panel" }
    },
  },
  "tts": { "vendor": "...", "params": { "voice_setting": /* voice A */ } }
}
```

Field availability evolves — confirm the current `vad` and `advanced_features` options against the live documentation during Phase 0, and note that the REST API's status codes and reason values were revised in May 2026.

Appendix B — Implementation recipes

The five mechanisms that are easy to get subtly wrong. Everything else in the build is ordinary application code.

B.1 Start and stop an agent

```
# start — returns {"agent_id": "..."}
POST https://api.agora.io/api/conversational-ai-agent/v2/projects/{APPID}/join
Authorization: Basic base64(CUSTOMER_ID:CUSTOMER_SECRET)
# body = the properties object from Appendix A

# stop — ALWAYS in a finally block; an orphaned agent bills until idle_timeout
POST ../v2/projects/{APPID}/agents/{agent_id}/leave

try:
    agent_ids = [join(role) for role in personas[:PANEL_SIZE]]
    await run_interview(session)
finally:
    for aid in agent_ids:
        leave(aid)          # never skip this — credits depend on it
```

B.2 The gateway endpoint — where silence is free

```
@app.post("/v1/{session}/{role}/chat/completions")
async def completions(session, role, req: Request):
    verify_secret(req)
    body = await req.json()

    stream = next_utterance(session, role, body["messages"])

    if stream is None:                # not this persona's turn
        return sse(["[DONE]"])        # no LLM call, no cost, no speech

    return StreamingResponse(sse(stream),
                             media_type="text/event-stream")

def sse(chunks):
    for c in chunks:
        if c == "[DONE]":
            yield "data: [DONE]\n\n"; return
        frame = {"choices": [{"index": 0, "delta": {"content": c}}]}
        yield f"data: {json.dumps(frame)}\n\n"
    yield "data: [DONE]\n\n"
```

TWO FAILURE MODES TO TEST ON DAY 1

Missing **[DONE]** leaves the agent waiting until it times out — the conversation appears to hang. A **leaked tool call** in the text stream gets read aloud as gibberish; strip tool-call deltas before forwarding and dispatch them to the conductor instead.

B.3 Floor control

```
def next_utterance(session, role, messages):
    s = STATE[session]

    if s.floor_holder != role:
        return None  # silent – the 3× quota saving

    ctx = s.shared_digest(last_n=8)  # what the OTHER personas heard
    return providers.stream(
        system = persona_prompt(role, s.difficulty, ctx),
        messages = messages[-10:],
        tools = CONDUCTOR_TOOLS,
    )

# after each candidate turn, before the next request arrives:
def decide_floor(s, analysis):
    if s.pending_handoff: return s.pending_handoff  # tool call
    if analysis.contradiction: return s.asker_of(analysis.turn) # challenge
    if analysis.no_business_framing and s.floor_holder == "technical":
        return "product"  # PS11 scenario
    if s.consecutive_turns >= 3: return s.next_round_robin()
    return s.floor_holder
```

B.4 Mock replay harness — the budget saver

```
# mock/replay.py – drives the gateway exactly as Agora would, at zero agent-minutes
SCRIPT = [
    ("technical", "I'd shard by tenant id and keep a routing table in Redis."),
    ("technical", "Latency dropped from 400ms to about 60ms."),
    ("technical", "Actually it was closer to 200ms."), # contradiction bait
    ("behavioural", "It depends, generally you just align stakeholders."),
]

for role, answer in SCRIPT:
    for p in personas:  # all 3 call, like Agora does
        r = post(f"/v1/test/{p}/chat/completions",
            messages=history + [{"role": "user", "content": answer}])
        if r.spoke: print(f"[{p}] {r.text}")
    assert_one_speaker(); history.append(...)
```

Seed the script with deliberate bait — a contradiction, a vague non-answer, a technically strong answer with no customer framing — so the hard capabilities are exercised on every run without burning agent-minutes.

B.5 Contradiction detection — narrow the task

This is the subtlest capability on a free-tier model. Comparing a new claim against the entire ledger produces false positives; comparing within a topic produces usable results.

```
# 1. extract, per turn – keep the schema tiny
{"claims": [{"claim": "...", "topic": "...", "quote": "..."}]}

# 2. compare ONLY within the same topic, and force both quotes
candidates = [c for c in ledger if c.topic == new.topic]
prompt = """Do these two statements from the same candidate conflict?
Answer only if they cannot both be true. Quote both.
Return {"conflict": bool, "quote_a": "", "quote_b": "", "why": ""}"""

# 3. on a hit → flag_contradiction(turn_a, turn_b) → floor to the original asker
```

Appendix C — Environment and configuration

AGORA_APP_ID	Project App ID
AGORA_APP_CERTIFICATE	For RTC token minting
AGORA_CUSTOMER_ID	REST authentication
AGORA_CUSTOMER_SECRET	REST authentication
GROQ_API_KEY	Utterance path — primary provider
CEREBRAS_API_KEY	Utterance path — first failover
OPENROUTER_API_KEY	Utterance path — second failover
GEMINI_API_KEY	Analysis path only — kept off the voice path deliberately
GATEWAY_SHARED_SECRET	Verified against the <code>api_key</code> Agora presents to your endpoint
GATEWAY_PUBLIC_URL	Base URL written into each agent's <code>llm.url</code>
PANEL_SIZE	<code>2</code> or <code>3</code> — the credit-pressure release valve (§5.2)
AGORA_IDLE_TIMEOUT	Keep low; orphaned agents bill until they time out

Appendix D — Suggested panel composition

PERSONA	ROLE	CONCERN, AND WHAT MAKES THEM INTERRUPT
Priya	Technical interviewer	Correctness, depth, trade-offs. Interrupts on hand-waving over mechanism. Accepts sound implementations without probing commercial impact — which is precisely what triggers the handoff to Arjun.
Arjun	Product manager	User impact, prioritisation, cost of being wrong. Challenges technically correct answers that never mention a customer. Owns the PS11 example scenario.
Meera	Behavioural interviewer	Collaboration, conflict, ownership. Runs role-play scenarios and holds candidates to specifics when answers drift into generalities.

Three is the right number: enough for genuine cross-role dynamics, few enough to keep voices distinguishable, credits manageable, and the floor-control logic demonstrable. A fourth persona adds cost and confusion without adding a scored capability.

Appendix E — Sources

- EchoSphere Hackathon 2026 — Problem Statement (PS11), About the Hackathon, and Official Participant Guidelines
- Agora Conversational AI — Start an agent, Update agent configuration, Broadcast a message using TTS, Send a custom instruction, Interrupt the agent mid-response, Event notifications, Release notes docs.agora.io/en/conversational-ai
- Agora Conversational AI pricing — \$0.10 per agent-minute, 300 free minutes per month, bundled ASR/LLM/TTS and bring-your-own-key terms docs.agora.io/en/conversational-ai/overview/pricing
- Groq free-tier rate limits (30 RPM; per-model daily request and token ceilings) and OpenAI-compatible endpoint documentation
- Google Gemini API free-tier rate limits by model (requests per minute, tokens per minute, requests per day)
- Repository audit — [AI-Interview](#) @ `f03c143`, 19 August 2026

